
Application Note: AN00131

USB CDC-ECM Class for Ethernet over USB

This application note shows how to create a USB device compliant to the standard USB Communications Device Class (CDC) and the Ethernet Control Model (ECM) Subclass on an Xmos multicore microcontroller.

The code associated with this application note provides an example of using the Xmos USB Device Library (XUD) and associated USB class descriptors to provide a framework for the creation of a USB device emulating Ethernet.

This example USB CDC-ECM implementation provides an emulated Ethernet interface running over high speed USB. It supports the standard requests associated with ECM model of the USB CDC specification.

The demo application handles the Ethernet frames received from the USB endpoints and hosts a HTTP web server acting as another virtual network device. A standard web browser from host PC can open the web page served from the USB device. The web page provides a statistics of different packets like ICMP, TCP, UDP etc received through the Ethernet frames from the host PC. This demonstrates a simple way in which Ethernet over USB applications can easily be deployed using an xCORE-USB device.

The demo application code can be extended to bridge an actual Ethernet interface by adding MAC and MII software layers. This enables you to create USB to Ethernet Adaptors using xCORE-USB device.

Note: This application note provides a standard USB CDC-ECM class device and as a result does not require external drivers to run on Linux and Mac. Windows doesn't support USB ECM model natively and thus requires third party drivers.

Required tools and libraries

- xTIMEcomposer Tools - Version 14.0.0
- Xmos USB library - Version 3.1.0

Required hardware

This application note is designed to run on an Xmos xCORE-USB series device.

The example code provided with the application has been implemented and tested on the xCORE-USB sliceKIT (XK-SK-U16-ST) but there is no dependency on this board and it can be modified to run on any development board which uses an xCORE-USB series device.

Prerequisites

- This document assumes familiarity with the Xmos xCORE architecture, the Universal Serial Bus 2.0 Specification and related specifications, the Xmos tool chain and the xC language. Documentation related to these aspects which are not specific to this application note are linked to in the references appendix.
- For descriptions of Xmos related terms found in this document please see the Xmos Glossary¹.
- For the full API listing of the Xmos USB Device (XUD) Library please see the document Xmos USB Device (XUD) Library².
- For information on designing USB devices using the XUD library please see the Xmos USB Device Design Guide for reference³.

¹<http://www.xmos.com/published/glossary>

²<http://www.xmos.com/published/xuddg>

³<http://www.xmos.com/published/xmos-usb-device-design-guide>

1 Overview

1.1 Introduction

USB Communications Class is a USB standard that defines mechanism for connecting Networking devices to host machines. USB CDC has specified multiple Subclass standards to support different networking devices. For Ethernet-style networking over USB, one of the following Subclass specifications is used:

1. Ethernet Control Model (ECM).
2. Ethernet Emulation Model (EEM).
3. Network Control Model (NCM).

Microsoft's RNDIS is also a famous Ethernet over USB standard that is not only supported in Windows but also in other OS platforms through thrird party drivers.

Ethernet over USB provides the following advantages:

- Application independent exchange of data over USB.
- Leverage the networking protocol stack present in operating systems.
- Abstraction for USB application developers at network level like sockets, HTTP (web pages) etc.
- USB-to-Ethernet Adaptors etc.

In this application note, the USB CDC-ECM class is chosen for the implementation of Ethernet emulation of USB device and it is discussed in detail which will help you to develop your own USB CDC-ECM product using xCORE-USB device or acts as a reference to implement other Ethernet supporting subclass on xCORE-USB device.

The standard USB CDC-ECM class is specified in a document **ECM 120.pdf** which can be found in the USB-IF website. (http://www.usb.org/developers/docs/devclass_docs/CDC1.2_WMC1.1_012011.zip)

1.2 Block diagram

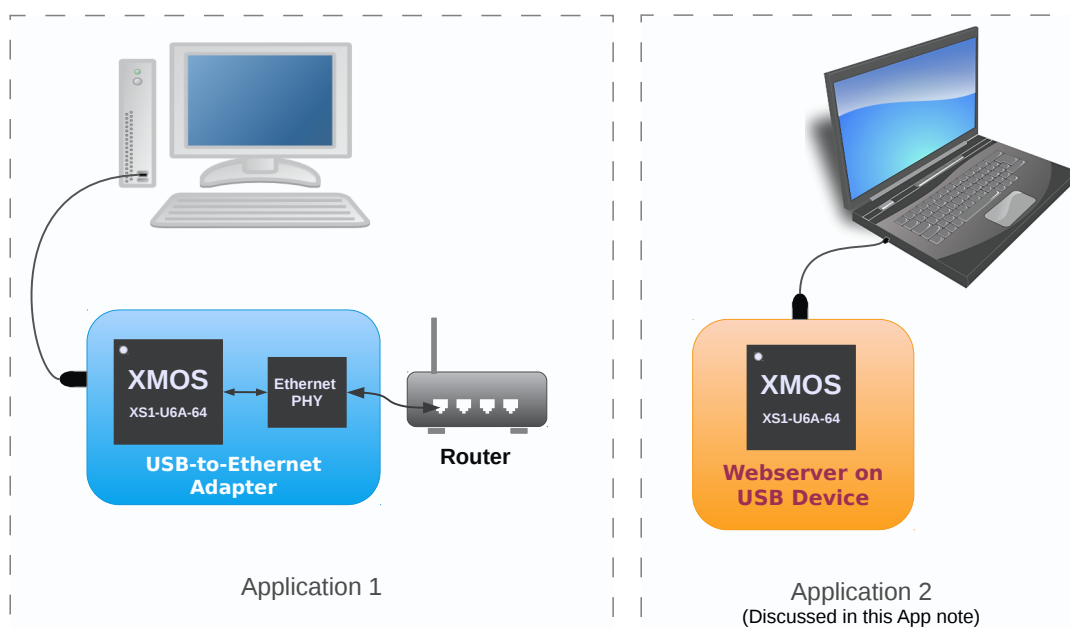


Figure 1: Block diagram of USB CDC-ECM applications

2 USB CDC-ECM Class application note

The example in this application note uses the XMOS USB device library and shows a simple program that enumerates a USB CDC-ECM Class device. The host side driver of this device will emulate an Ethernet interface that seamlessly connects with the network stack of host's operating system. The USB device runs a very simple web server and acts like another device on Ethernet network i.e. the USB device imitates a network interface card of the host machine and a server connected to the host machine through Ethernet network.

For this USB CDC-ECM device application example, the system comprises four tasks running on separate logical cores of an xCORE-USB multicore microcontroller. bao gồm

The tasks perform the following operations.

- A task containing the USB library functionality to communicate over USB.
- A task implementing Endpoint0 responding to both standard and CDC-ECM class-specific USB requests.
- A task implementing the data endpoints and notification endpoint of the CDC-ECM class. It handles tx and rx buffers and provides APIs for applications to receive and transmit Ethernet frames.
- A task containing Ethernet frame handler and simple HTTP server.

These tasks communicate via the use of xCONNECT channels which allow data to be passed between application code running on separate logical cores. In this example, XC interfaces are used, which abstracts out the channel communication details with function level interface.

The following diagram shows the task and communication structure for this USB CDC-ECM class application example.

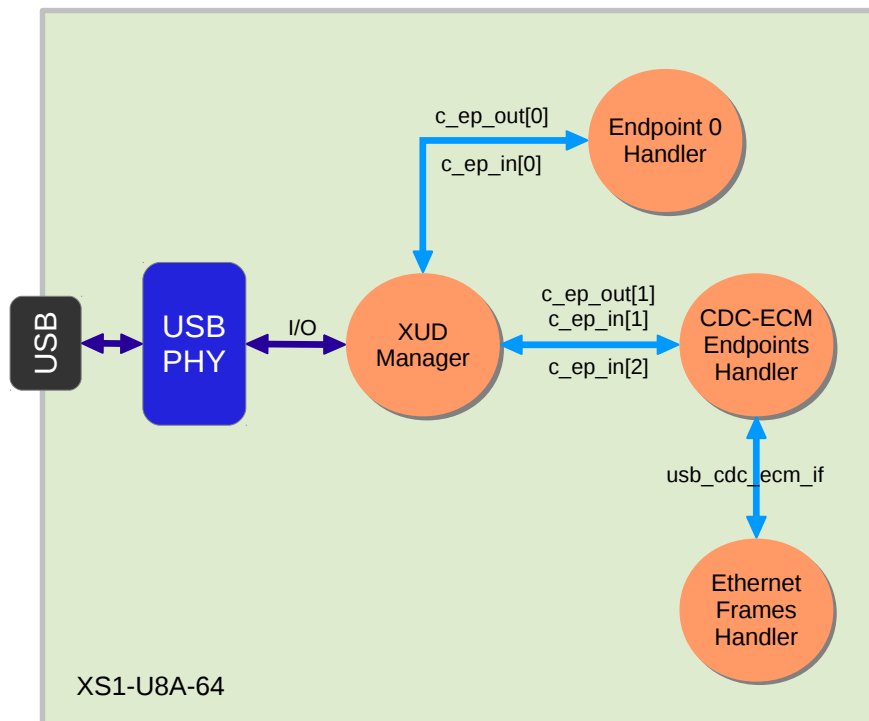


Figure 2: Task diagram of the USB CDC-ECM example

2.1 Makefile additions for this example

To start using the USB library, you need to add **lib_usb** to your makefile:

```
USED_MODULES = ... lib_usb ...
```

You can then access the USB functions in your source code via the *usb.h* header file:

```
#include <usb.h>
```

2.2 Source code files

The example application consists of several source code files and the following list provides an overview of how the source code is organized.

- *xud_ecm.xc*, *xud_ecm.h* - Contains the USB CDC-ECM implementation which includes the USB descriptors, endpoints handler tasks (functions), class-specific defines, xC interface to read/write Ethernet frames.
- *main.xc* - Contains main() function and some USB defines.
- *packet_buffer.xc*, *queue.xc* - Buffer implementation to hold Ethernet frames (Max of 1514 bytes) and to allocate and deallocate dynamically from buffer pool. Queue implementation to queue up the tx and rx frame buffers in a FIFO fashion.
- *ethernet.xc* - Contains implementation of application handling the Ethernet frames and upper layer protocols like ICMP, DNS etc
- *eth_tcp* - This folder has TCP/IP and HTTP server implementation source files. Note: This is not a complete TCP/IP stack.

2.3 Declaring resource and setting up the USB components

main.xc has some defines in it that are used to configure the XMOS USB device library. These are displayed below.

```
/* USB Endpoint Defines */
#define XUD_EP_COUNT_OUT 2 //Includes EP0 (1 OUT EP0 + 1 BULK OUT EP)
#define XUD_EP_COUNT_IN 3 //Includes EP0 (1 IN EP0 + 1 INTERRUPT IN EP + 1 BULK IN EP)
```

The above set of defines describe the endpoints configuration for this device. This example has bi-directional communication with the host machine via the standard endpoint0 and three other endpoints for implementing the part of our CDC-ECM class.

All these defines are passed to the setup function for the USB library which is called from **main()**.

2.4 The application main() function

Below is the source code for the main function of this application, which is taken from the source file *main.xc*


```
int main() {  
    chan c_ep_out[XUD_EP_COUNT_OUT], c_ep_in[XUD_EP_COUNT_IN];  
    interface usb_cdc_ecm_if cdc_ecm;  
  
    /* 'Par' statement to run the following tasks in parallel */  
    par  
    {  
        on USB_TILE: xud(c_ep_out, XUD_EP_COUNT_OUT, c_ep_in, XUD_EP_COUNT_IN,  
                        null, XUD_SPEED_HS, XUD_PWR_SELF);  
  
        on USB_TILE: Endpoint0(c_ep_out[0], c_ep_in[0]);  
  
        on USB_TILE: CdcEcmEndpointsHandler(c_ep_in[1], c_ep_out[1], c_ep_in[2], cdc_ecm);  
  
        on USB_TILE: EthernetFrameHandler(cdc_ecm);  
    }  
    return 0;  
}
```

Looking at this in a more detail you can see the following:

- The par statement starts four separate tasks in parallel.
- There is a task to configure and execute the USB library: `xud()`. This library call runs in an infinite loop and handles all the underlying USB communications and provides abstraction at the endpoints level.
- There is a task to startup and run the Endpoint0 code: `Endpoint0()`. It handles the control endpoint zero and must be run in a separate logical core in order to promptly respond to the control requests from host.
- There is a task to handle all the other three endpoints required for the CDC-ECM class: `CdcEcmEndpointsHandler()`. This function handles one bulk OUT and one bulk IN endpoints for Ethernet frame transmissions and one interrupt IN endpoint for sending notifications to host.
- There is a task to handle the Ethernet frames received from the host: `EthernetFrameHandler()`. This task implements network stack and hosts the HTTP server. It also handles DNS queries, DHCP requests and ICMP pings.
- The define `USB_TILE` describes the tile on which the individual tasks will run.
- In this example all tasks run on the same tile as the USB PHY although this is only a requirement of `xud()`.
- The xCONNECT communication channels and the xC interface `cdc_ecm` used for inter-task communication are setup at the beginning of `main()` and passed on to respective tasks.
- The USB defines discussed earlier are passed into the function `xud()`.

2.5 Configuring the USB Device ID

The USB ID values used for vendor ID, product ID and device version number are defined in the file `xud_ecm.xc`. These are used by the host machine to determine the vendor of the device (in this case Xmos) and the product plus the firmware version.

```
/* USB CDC device product defines */  
#define BCD_DEVICE 0x0100  
#define VENDOR_ID 0x20B1  
#define PRODUCT_ID 0x0402
```

2.6 USB Descriptors

USB CDC class device has to support class-specific descriptors apart from the standard descriptors defined in the USB specifications. These class specific descriptors are customized according to the need of the USB CDC device. In the example application code, the descriptors implement the ECM model of the CDC class to support Ethernet emulation at the host machine.

The following figure shows the descriptors used in the example code.

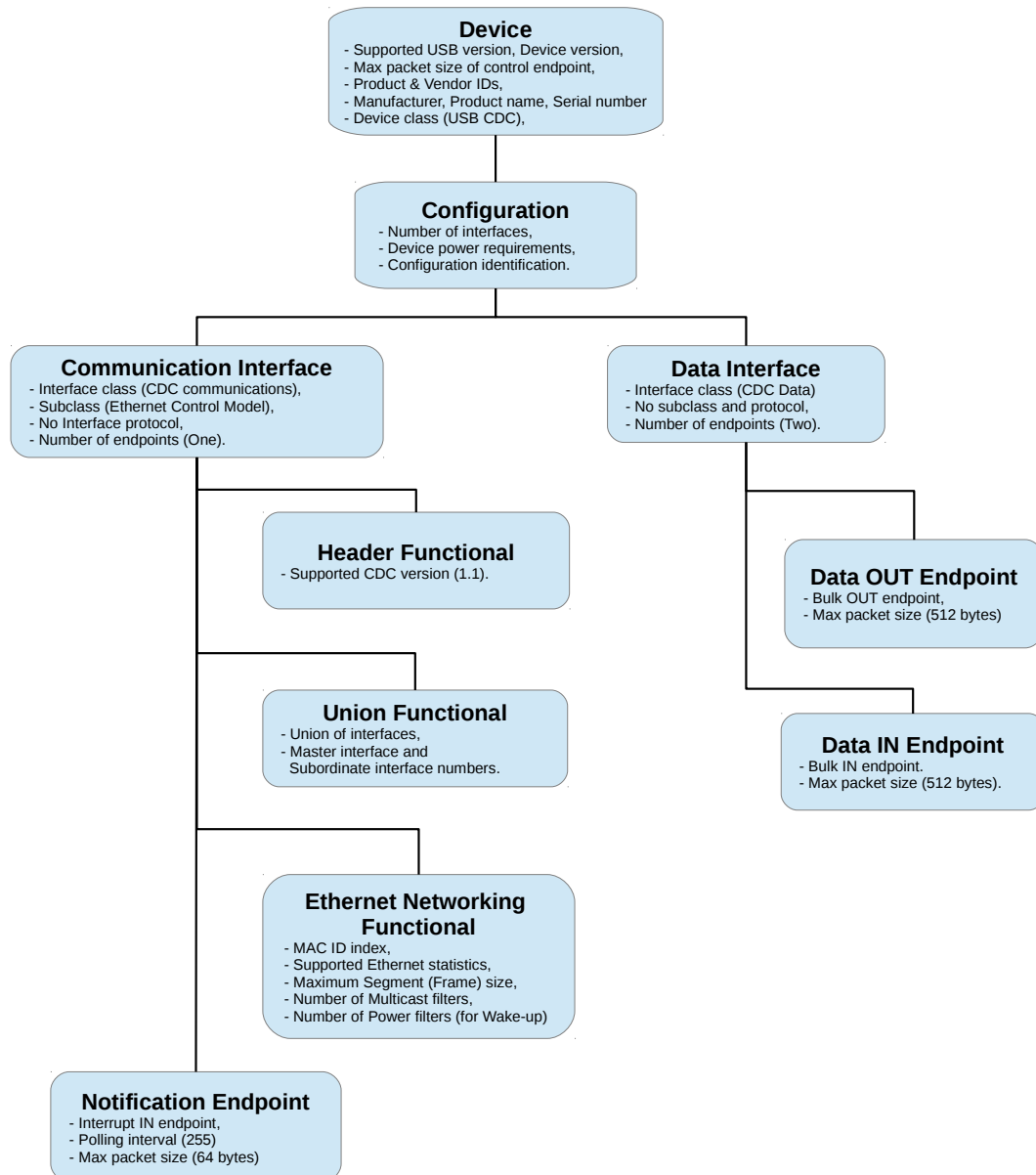


Figure 3: USB descriptors hierarchical structure of CDC-ECM example

2.6.1 USB Device Descriptor

xud_ecm.xc is where the standard USB device descriptor is declared for the CDC-ECM class device. Below is the structure which contains this descriptor. This will be requested by the host when the device is enumerated on the USB bus.

```
/* USB Device Descriptor */
static unsigned char devDesc[] =
{
    0x12,          /* 0 bLength */
    USB_DESCTYPE_DEVICE, /* 1 bdescriptorType - Device */
    0x00,          /* 2 bcdUSB version */
    0x02,          /* 3 bcdUSB version */
    USB_CLASS_COMMUNICATIONS, /* 4 bDeviceClass - USB CDC Class */
    0x00,          /* 5 bDeviceSubClass - Specified by interface */
    0x00,          /* 6 bDeviceProtocol - Specified by interface */
    0x40,          /* 7 bMaxPacketSize for EP0 - max = 64 */
    (VENDOR_ID & 0xFF), /* 8 idVendor */
    (VENDOR_ID >> 8), /* 9 idVendor */
    (PRODUCT_ID & 0xFF), /* 10 idProduct */
    (PRODUCT_ID >> 8), /* 11 idProduct */
    (BCD_DEVICE & 0xFF), /* 12 bcdDevice */
    (BCD_DEVICE >> 8), /* 13 bcdDevice */
    0x01,          /* 14 iManufacturer - index of string */
    0x02,          /* 15 iProduct - index of string */
    0x00,          /* 16 iSerialNumber - index of string */
    0x01          /* 17 bNumConfigurations */
};
```

From this descriptor you can see that product, vendor and device firmware revision are all coded into this structure. This will allow the host machine to recognise the CDC device when it is connected to the USB bus.

2.6.2 USB Configuration Descriptor

The USB configuration descriptor is used to configure the device in terms of the device class and the endpoints setup. The hierarchy of descriptors under a configuration includes interfaces descriptors, class-specific descriptors and endpoints descriptors.

when a host requests a configuration descriptor, the entire configuration hierarchy including all the related descriptors are returned to the host. The following code shows the configuration hierarchy of the demo application.

```
/* USB Configuration Descriptor */
static unsigned char cfgDesc[] = {

    0x09, /* 0 bLength */
    USB_DESCRIPTOR_CONFIGURATION, /* 1 bDescriptorType - Configuration */
    0x47, 0x00, /* 2 wTotalLength */
    0x02, /* 4 bNumInterfaces */
    0x01, /* 5 bConfigurationValue */
    0x04, /* 6 iConfiguration - index of string */
    0x80, /* 7 bmAttributes - Bus powered */
    0xC8, /* 8 bMaxPower - 400mA */

    /* CDC Communication interface */
    0x09, /* 0 bLength */
    USB_DESCRIPTOR_INTERFACE, /* 1 bDescriptorType - Interface */
    0x00, /* 2 bInterfaceNumber - Interface 0 */
    0x00, /* 3 bAlternateSetting */
    0x01, /* 4 bNumEndpoints */
    USB_CLASS_COMMUNICATIONS, /* 5 bInterfaceClass */
    USB_CDC_ECM_SUBCLASS, /* 6 bInterfaceSubClass - Ethernet Control Model */
    0x00, /* 7 bInterfaceProtocol - No specific protocol */
    0x00, /* 8 iInterface - No string descriptor */

    /* Header Functional descriptor */
    0x05, /* 0 bLength */
    USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
    0x00, /* 2 bDescriptorSubtype, HEADER */
    0x10, 0x01, /* 3 bcdCDC */

    /* Union Functional descriptor */
    0x05, /* 0 bLength */
    USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
    0x06, /* 2 bDescriptorSubtype, UNION */
    0x00, /* 3 bControlInterface - Interface 0 */
    0x01, /* 4 bSubordinateInterface0 - Interface 1 */

    /* Ethernet Networking Functional descriptor */
    0x0D, /* 0 bLength - 13 bytes */
    USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
    0x0F, /* 2 bDescriptorSubtype, ETHERNET NETWORKING */
    0x03, /* 3 iMACAddress, Index of MAC address string */
    0x00, 0x00, 0x00, 0x00, /* 4 bmEthernetStatistics - Handles None */
    0xEA, 0x05, /* 8 wMaxSegmentSize - 1514 bytes */
    0x00, 0x00, /* 10 wNumberMCFilters - No multicast filters */
    0x00, /* 12 bNumberPowerFilters - No wake-up feature */

    /* Notification Endpoint descriptor */
    0x07, /* 0 bLength */
    USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
    (CDC_NOTIFICATION_EP_NUM | 0x80), /* 2 bEndpointAddress - IN endpoint */
    0x03, /* 3 bmAttributes - Interrupt type */
    0x40, /* 4 wMaxPacketSize - Low */
    0x00, /* 5 wMaxPacketSize - High */
    0xFF, /* 6 bInterval */

    /* CDC Data interface */
    0x09, /* 0 bLength */
    USB_DESCRIPTOR_INTERFACE, /* 1 bDescriptorType */
    0x01, /* 2 bInterfaceNumber */
    0x00, /* 3 bAlternateSetting */
    0x02, /* 4 bNumEndpoints */
    USB_CLASS_CDC_DATA, /* 5 bInterfaceClass */
    0x00, /* 6 bInterfaceSubClass */
}
```

```

0x00,          /* 7 bInterfaceProtocol*/
0x00,          /* 8 iInterface - No string descriptor*/

/* Data OUT Endpoint descriptor */
0x07,          /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
CDC_DATA_RX_EP_NUM, /* 2 bEndpointAddress - OUT endpoint */
0x02,          /* 3 bmAttributes - Bulk type */
0x00,          /* 4 wMaxPacketSize - Low */
0x02,          /* 5 wMaxPacketSize - High */
0x00,          /* 6 bInterval */

/* Data IN Endpoint descriptor */
0x07,          /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
(CDC_DATA_TX_EP_NUM | 0x80), /* 2 bEndpointAddress - IN endpoint */
0x02,          /* 3 bmAttributes - Bulk type */
0x00,          /* 4 wMaxPacketSize - Low byte */
0x02,          /* 5 wMaxPacketSize - High byte */
0x01,          /* 6 bInterval */
};

```

The configuration descriptor tells host about the power requirements of the device and the number of interfaces it supports.

The interface descriptors describe on how the host should communicate with the device in the class level. There are two interface descriptors in a USB CDC-ECM device.

The **CDC Communication interface** descriptor is for device management. You can see from the code that the device uses Ethernet Control Model as the interface subclass, this will make hosts to load default driver for CDC Ethernet. This interface has subordinate descriptors like CDC functional descriptors and a notification endpoint descriptor. The class-specific functional descriptors are discussed in detail in the next section. The notification endpoint is an interrupt IN endpoint and is used to report device's network connection state to the host. This endpoint is not used in this example application but will be employed when bridging an Ethernet interface to the USB-ECM device.

The **CDC Data interface** descriptor defines the interface for Ethernet frames transmission and reception between host and device. This interface has two endpoints, one bulk OUT endpoint for data transmissions from host to device and one bulk IN endpoint for data transmissions from device to host.

```

/* String table - unsafe as accessed via shared memory */
static char * unsafe stringDescriptors[] =

```

The above code from the endpoint descriptors shows that the maximum packet size of these endpoints to be 512 bytes (0x200) which is suited for handling Ethernet frames in less number of transactions.

2.6.3 USB CDC-ECM Functional Descriptors

Functional descriptors describe the content of class-specific information within the Communication Class interface. The 'USB_DESCRIPTOR_CS_INTERFACE' define is used in the descriptor structures to identify them. There are three functional descriptors used in this CDC-ECM example. They are:

1. Header Functional Descriptor.
2. Union Functional Descriptor.
3. Ethernet Networking Functional Descriptor.

The Header and Union functional descriptors are generally used in all CDC subclasses and the Ethernet Networking functional descriptor is meant for the ECM subclass.

Header functional descriptor mentions the version of the CDC specification the interface comply with and it is shown below as found in the *cfgDesc[]* structure.

```
0x00,0x00,0x00,0x00,    /* 4 bmEthernetStatistics - Handles None */
0xEA,05,                /* 8 wMaxSegmentSize - 1514 bytes */
0x00,0x00,              /* 10 wNumberMCFilters - No multicast filters */
```

Note: The CDC version number (1.10) is mentioned as BCD in little endian format.

Union functional descriptor groups the interfaces that forms a CDC functional unit. It specifies one of the interfaces as master to handle control messages of the unit. Following code from the CDC-ECM example shows that the the Communication Class interface 0 acts as master and the Data Class interface 1 acts as subordinate and together forming a single functional unit.

```
0x07,                    /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
(CDC_NOTIFICATION_EP_NUM | 0x80), /* 2 bEndpointAddress - IN endpoint */
0x03,                    /* 3 bmAttributes - Interrupt type */
```

Ethernet networking functional descriptor provides the Ethernet related capabilities and parameters to the host. Following code shows the fields of the descriptor.

```
/* CDC Data interface */
0x09,                    /* 0 bLength */
USB_DESCRIPTOR_INTERFACE, /* 1 bDescriptorType */
0x01,                    /* 2 bInterfaceNumber */
0x00,                    /* 3 bAlternateSetting */
0x02,                    /* 4 bNumEndpoints */
```

The above code shows the followings:

- 48-bit MAC address is provided through the string descriptor index, the string descriptor will carry the MAC address in a Unicode string format (example: 00229708A003).
- Ethernet statistics which the device collects. All bits are set to '0' in this example denoting that the device will not collect any Ethernet statistics.
- Maximum segment size of 1514 bytes for handling 802.3 Ethernet frames.
- Ethernet multicast filters and power filters are not supported in this USB-ECM example.

2.6.4 USB String Descriptors

String descriptors provide human readable information for your device and you can configure them with your USB product information. In CDC-ECM class it also provides the 48-bit MAC address of the device. The descriptors are placed in an array as shown in the below code.

```
/* String table - unsafe as accessed via shared memory */
static char * unsafe stringDescriptors[]=
{
    "\x09\x04",          /* Language ID string (US English) */
    "Xmos",              /* iManufacturer */
    "Xmos USB CDC Ethernet Device", /* iProduct */
    "002297000000",      /* iMACAddress */
    "Config",            /* iConfiguration string */
};
```

The Xmos USB library will take care of encoding the strings into Unicode and structures the content into USB string descriptor format.

2.7 USB Standard and Class-Specific requests

In *xud_ecm.xc* there is a function *Endpoint0()* which handles all the USB control requests sent by host to control endpoint 0. USB control requests includes both standard USB requests and the CDC-ECM class-specific requests.

In *Endpoint0()* function, a USB request is received as a setup packet by calling *USB_GetSetupPacket()* library function. The setup packet structure is then examined to distinguish between standard and class-specific requests.

The Xmos USB library provides a function *USB_StandardRequests()* to handle all the standard USB requests. This function is called with setup packet and descriptors structures as shown below

```
/* Returns  XUD_RES_OKAY if handled okay,
 *          XUD_RES_ERR if request was not handled (STALled),
 *          XUD_RES_RST for USB Reset */
unsafe{
result = USB_StandardRequests(ep0_out, ep0_in, devDesc,
                             sizeof(devDesc), cfgDesc, sizeof(cfgDesc),
                             null, 0, null, 0, stringDescriptors, sizeof(stringDescriptors)/sizeof(stringDescriptors[0]),
                             sp, usbBusSpeed);
}
```

The CDC Communication interface uses endpoint 0 as management element and receives class-specific control requests on it. The following code shows how the ECM class-specific requests are filtered and passed to a function *ControlInterfaceClassRequests()* for further handling.

```
switch(bmRequestType)
{
    /* Direction: Device-to-host and Host-to-device
     * Type: Class
     * Recipient: Interface
     */
    case USB_BMREQ_H2D_CLASS_INT:
    case USB_BMREQ_D2H_CLASS_INT:

        /* Inspect for CDC Communications Class interface num */
        if(sp.wIndex == 0)
        {
            /* Returns  XUD_RES_OKAY if handled,
             *          XUD_RES_ERR if not handled,
             *          XUD_RES_RST for bus reset */
            result = ControlInterfaceClassRequests(ep0_out, ep0_in, sp);
        }
        break;
}
```

The *ControlInterfaceClassRequests()* function is the place to handle all CDC-ECM requests, These requests are defined in the *xud_ecm.xc* as follows:

```
/* CDC ECM Class requests Section 6.2 in CDC ECM spec */
#define SET_ETHERNET_MULTICAST_FILTERS          0x40
#define SET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER 0x41
#define GET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER 0x42
#define SET_ETHERNET_PACKET_FILTER             0x43
#define GET_ETHERNET_STATISTIC                  0x44
/* 45h-4Fh RESERVED (future use) */
```

According to ECM specification, except *SET_ETHERNET_PACKET_FILTER* all the other requests are optional. These requests are not used in this example application but will be used when bridging an Ethernet

interface to the USB device. In case of bridging an Ethernet interface, the Ethernet MAC software layer and PHY software layer will be employed and the MAC API functions can be directly used to perform the actions requested by the class-specific requests.

2.8 Ethernet frame handling

The Data Class interface containing the bulk endpoints is used for exchanging Ethernet frames between the device and host. The Ethernet frame includes everything starting from the destination MAC address to the end of the data field, but excludes the CRC. This frame structure is shown below:

Destination MAC Address (6 bytes)	Source MAC Address (6 bytes)	Type (2 bytes)	Data (46 – 1500 bytes)
--------------------------------------	---------------------------------	-------------------	---------------------------

Figure 4: Ethernet frame exchanged over the USB endpoints

To handle asynchronous communication over two endpoints, events are used by means of select statements as shown in the following piece of code from *CdcEcmEndpointsHandler()* task.

```
select
{
    case XUD_GetData_Select(c_epbulk_out, epbulk_out, length, result):
        if(result == XUD_RES_OKAY)
        {
            /* Received some data */
            if(length < MAX_EP_SIZE) {
                /* USB Short packet or Zero length packet is received */
                outLen += length;
                /* Ethernet packet is received completely */
                qPut(toDevQ, outBufId, outLen);

                if(qIsFull(toDevQ)) {
                    devWaiting = 1;
                } else {
                    outBufId = packetBufferAlloc();
                    outLen = 0;
                    XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[]));
                }
            } else {
                /* USB Full packet */
                outLen += MAX_EP_SIZE;
                XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[])+outLen);
            }
        } else {
            XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[])+outLen);
        }
        break;

    case XUD_SetData_Select(c_epbulk_in, epbulk_in, result):
        /* USB Packet sent successfully when result is XUD_RES_OKAY */
        int index = qPeek(toHostQ);
        int bytesSent = toHostQ.data[index].from - inIndex;
        inIndex = toHostQ.data[index].from;
        int bytesToSend = toHostQ.data[index].len - toHostQ.data[index].from;

        if(bytesToSend) {
            if (bytesToSend > MAX_EP_SIZE) {
                /* Still Large packet, split the transfer */
                bytesToSend = MAX_EP_SIZE;
            }
        }
    }
```

```

        XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[])+inIndex, bytesToSend);
        toHostQ.data[index].from += bytesToSend;

    } else if (bytesSent == MAX_EP_SIZE){
        /* Send a Zero Length Packet to indicate completion of transfer */
        XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[]), 0);
        inIndex += bytesSent;
    } else {
        /* Ethernet frame transfer is over */
        packetBufferFree(toHostQ.data[index].packet);
        /* Remove packet out of queue */
        qGet(toHostQ);

        /* Check if other packet is waiting to be sent to host */
        if(!qIsEmpty(toHostQ)) {
            index = qPeek(toHostQ);
            inBufId = toHostQ.data[index].packet;
            inIndex = toHostQ.data[index].from;
            bytesToSend = toHostQ.data[index].len;

            if(bytesToSend > MAX_EP_SIZE) {
                bytesToSend = MAX_EP_SIZE;
            }
            XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[])+inIndex, bytesToSend);
            toHostQ.data[index].from += bytesToSend;
        } else {
            /* No packets are available to send */
            hostWaiting = 1;
        }
    }
}
break;

```

When OUT endpoint receives data, an event is triggered and the *XUD_GetData_Select()* case is executed. Similarly, when IN endpoint completes sending data to host the *XUD_SetData_Select()* case is executed.

The maximum size of an Ethernet frame (1514 bytes) is greater than the maximum packet size (512 bytes) of the USB endpoints, therefore an Ethernet frame may be split into multiple USB packet transfers. A USB short packet notifies the end of an Ethernet frame, if the frame size is exactly a multiple of the maximum packet size of USB then a zero length packet is used to notify the end of frame.

You can see from the above code that a free buffer is allocated using *packetBufferAlloc()*, which is used to receive an Ethernet frame. Once the frame is completely received the *qPut()* function is used to add the frame into a queue. The *QUEUE_LENGTH* (four) defined in *queue.h* determines the maximum number of ethernet frames buffered up in the queue.

There is also a separate queue to handle the Ethernet frames that are transmitted to the host from the device.

2.9 Application interface

The application interface is the set of functions defined as xC interface that enables application tasks to send/receive Ethernet frames over the ECM data endpoints. The API functions abstracts out all the buffering implementation details done at the endpoint level. This xC interface is declared in *xud_ecm.h* file as shown below.

```

interface usb_cdc_ecm_if {
    int is_frame_available();

    [[guarded]] void read_frame(unsigned char buf[], REFERENCE_PARAM(unsigned, length));
    [[guarded]] void send_frame(unsigned char buf[], REFERENCE_PARAM(unsigned, length));
};

```

In the above code, the *read_frame()* function gets an Ethernet frame which is waiting in the reception queue, Similarly, the *write_frame()* function adds the Ethernet frame to the transmission queue.

These interface functions pass arguments and return values over xCONNECT channels and provide well defined inter-task communication. The server side of these functions are defined under select case statements in the *CdcEcmEndpointsHandler()* task.

2.10 Demo application

In this USB CDC-ECM example, the Ethernet frames received from the host are handled by the *Ethernet-FrameHandler()* task. This application task runs a simple HTTP server acting as a virtual network device. This task performs the following:

- Handles DHCP requests from the host PC to provide an IP address to it. The IP addresses are defined in the *ethernet.xc* file as shown below

```
/* IP Addresses of the USB device (Server) and the Host PC (Client) */
int ipAddressServer    = 0xA9FE5555;
int ipAddressClient    = 0xA9FEAAAA;
unsigned char ipAddressServerArray[4] = {169, 254, 85, 85};
unsigned char ipAddressClientArray[4] = {169, 254, 170, 170};
```

The client IP address corresponds to the host PC and the server IP address belongs to the USB device.

- Handles DNS queries containing the server name defined in the *ethernet.xc* file. The server name is as shown below.

```
/* Server name as found in the DNS query */
unsigned char localName[] = "\x08xmos-cdc\x05local";
```

The *localName[]* is initialized in DNS name format and it corresponds to "xmos-cdc.local"

- Handles HTTP webpage requests and ICMP control ping requests.
- Collects a statistics of the different packets received from the host PC and embeds the information in the web page.

APPENDIX A - Demo Hardware Setup

To setup the demo hardware the following boards are required.

- xCORE-USB sliceKIT (XK-SK-U16-ST)
 - xCORE-USB Core board.
 - USB A/B sliceCARD.
 - xTAG-2 debug adaptor
 - Power supply

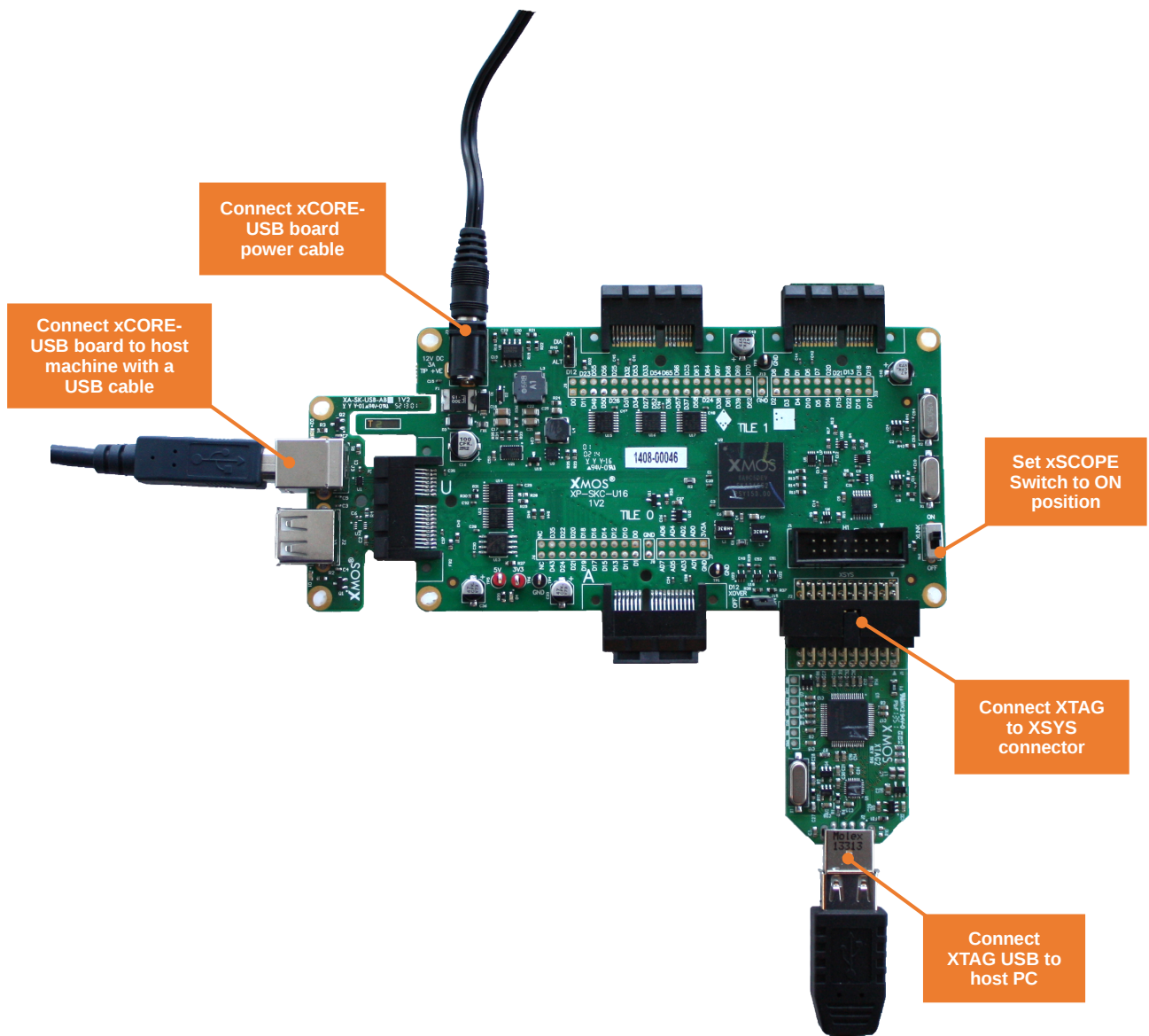


Figure 5: XMOS xCORE-USB sliceKIT

The hardware should be configured as displayed above for this demo:

- The XTAG debug adapter should be connected to the XSYS connector and the XTAG USB should be connected to the host machine.
- The USB sliceCARD should be connected to the U slot (J4 header) of the xCORE-USB Core board and the other end of USB sliceCARD should be connected to host machine using a USB cable.
- The XLINK switch on the core board should be set on ON position.
- The xCORE-USB core board should have the power cable connected.

Note: The XLINK switch is set to ON position to enable xSCOPE to function. The use of xSCOPE is required in this application so that the print messages that are generated on the device as part of the demo do not interfere with the real-time behavior of the USB device.

APPENDIX B - Launching the demo application

Once the demo example has been built either from the command line using `xmake` or via the build mechanism of xTIMEcomposer studio we can execute the application on the xCORE-USB sliceKIT.

Once built there will be a `bin` directory within the project which contains the binary for the xCORE device. The xCORE binary has a XMOS standard `.xe` extension.

B.1 Launching from the command line

From the command line the `xrun` tool is used to download code to both the xCORE devices. Changing into the `bin` directory of the project we can execute the code on the xCORE microcontroller as follows:

```
> xrun --xscope app_usb_ecm.xe          <-- Download and execute the xCORE code
```

Once this command has executed the CDC Ethernet device should have enumerated on your machine and you will see the following text in the console window:

```
--XMOS USB CDC-ECM Class demo--  
  
Server IP Address: 169.254.85.85  
Server URL: http://xmos-cdc.local
```

B.2 Launching from xTIMEcomposer Studio

From xTIMEcomposer Studio the run mechanism is used to download code to xCORE device. Select the xCORE binary from the `bin` directory, right click and then follow the instructions below.

- Select **Run As**.
- Select **Run Configurations**.
- Double click on xCORE application**.
- Enable xSCOPE in Target I/O options:

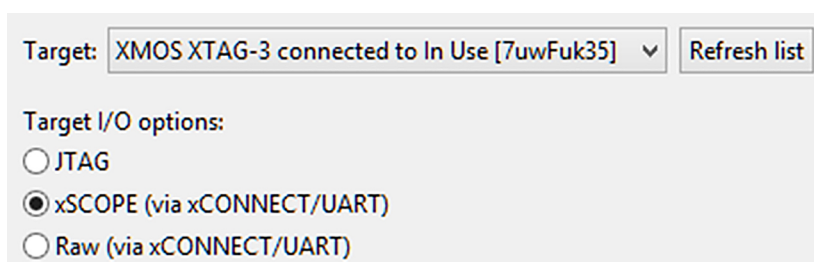


Figure 6: xTIMEcomposer xSCOPE configuration

- Click **Apply** and then **Run**.

Once the application is launched the CDC Ethernet device should have enumerated on your machine and you will see the following text in the console window:

```
--XMOS USB CDC-ECM Class demo--  
  
Server IP Address: 169.254.85.85  
Server URL: http://xmos-cdc.local
```

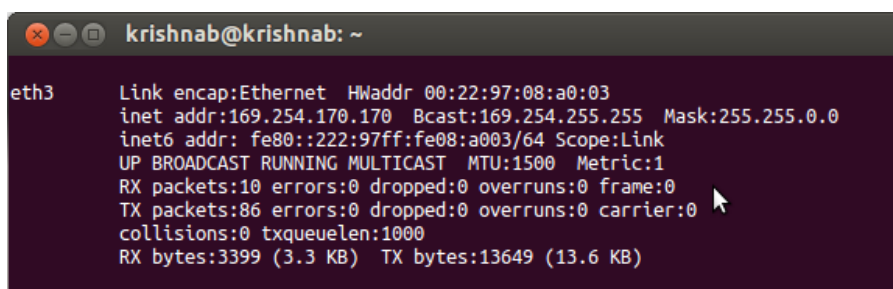
B.3 Running the demo

This demo works right away on platforms that have native support for USB CDC-ECM class. Most of the Linux OS have native support for this CDC Ethernet. The demo application is tested on Ubuntu 12.04 LTS version.

B.3.1 Running on Linux

Once the USB device is enumerated in the host machine, the default CDC Ethernet driver will be loaded. This host driver emulates the virtual Ethernet interface.

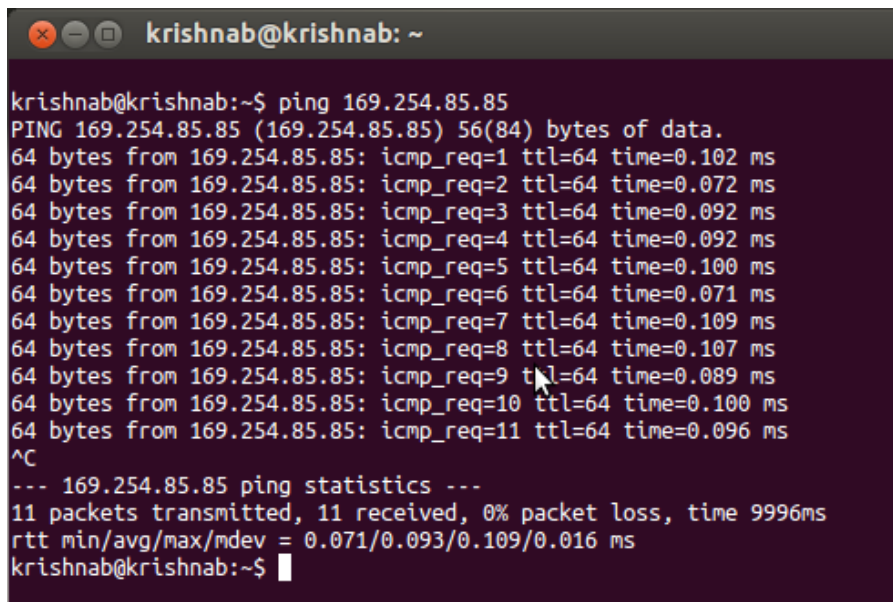
- Run the command *ifconfig* in a terminal to view the emulated Ethernet interface as shown in the following figure.



```
krishnab@krishnab: ~
eth3      Link encap:Ethernet  HWaddr 00:22:97:08:a0:03
          inet addr:169.254.170.170  Bcast:169.254.255.255  Mask:255.255.0.0
          inet6 addr: fe80::222:97ff:fe08:a003/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:86 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3399 (3.3 KB)  TX bytes:13649 (13.6 KB)
```

Figure 7: Emulated Ethernet interface in the host PC

- Run *ping 169.254.85.85* command to ping the server running in the USB device. This ping command sends ICMP request packets to the server and it is shown in the following figure



```
krishnab@krishnab: ~
krishnab@krishnab:~$ ping 169.254.85.85
PING 169.254.85.85 (169.254.85.85) 56(84) bytes of data.
64 bytes from 169.254.85.85: icmp_req=1 ttl=64 time=0.102 ms
64 bytes from 169.254.85.85: icmp_req=2 ttl=64 time=0.072 ms
64 bytes from 169.254.85.85: icmp_req=3 ttl=64 time=0.092 ms
64 bytes from 169.254.85.85: icmp_req=4 ttl=64 time=0.092 ms
64 bytes from 169.254.85.85: icmp_req=5 ttl=64 time=0.100 ms
64 bytes from 169.254.85.85: icmp_req=6 ttl=64 time=0.071 ms
64 bytes from 169.254.85.85: icmp_req=7 ttl=64 time=0.109 ms
64 bytes from 169.254.85.85: icmp_req=8 ttl=64 time=0.107 ms
64 bytes from 169.254.85.85: icmp_req=9 ttl=64 time=0.089 ms
64 bytes from 169.254.85.85: icmp_req=10 ttl=64 time=0.100 ms
64 bytes from 169.254.85.85: icmp_req=11 ttl=64 time=0.096 ms
^C
--- 169.254.85.85 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 996ms
rtt min/avg/max/mdev = 0.071/0.093/0.109/0.016 ms
krishnab@krishnab:~$
```

Figure 8: Pinging server from host PC

- Open the URL <http://xmos-cdc.local> in a standard web browser like Mozilla or Chrome to see the web page hosted by the USB device. The web page provides a statistics of the packets handled by the USB device and it is shown below:

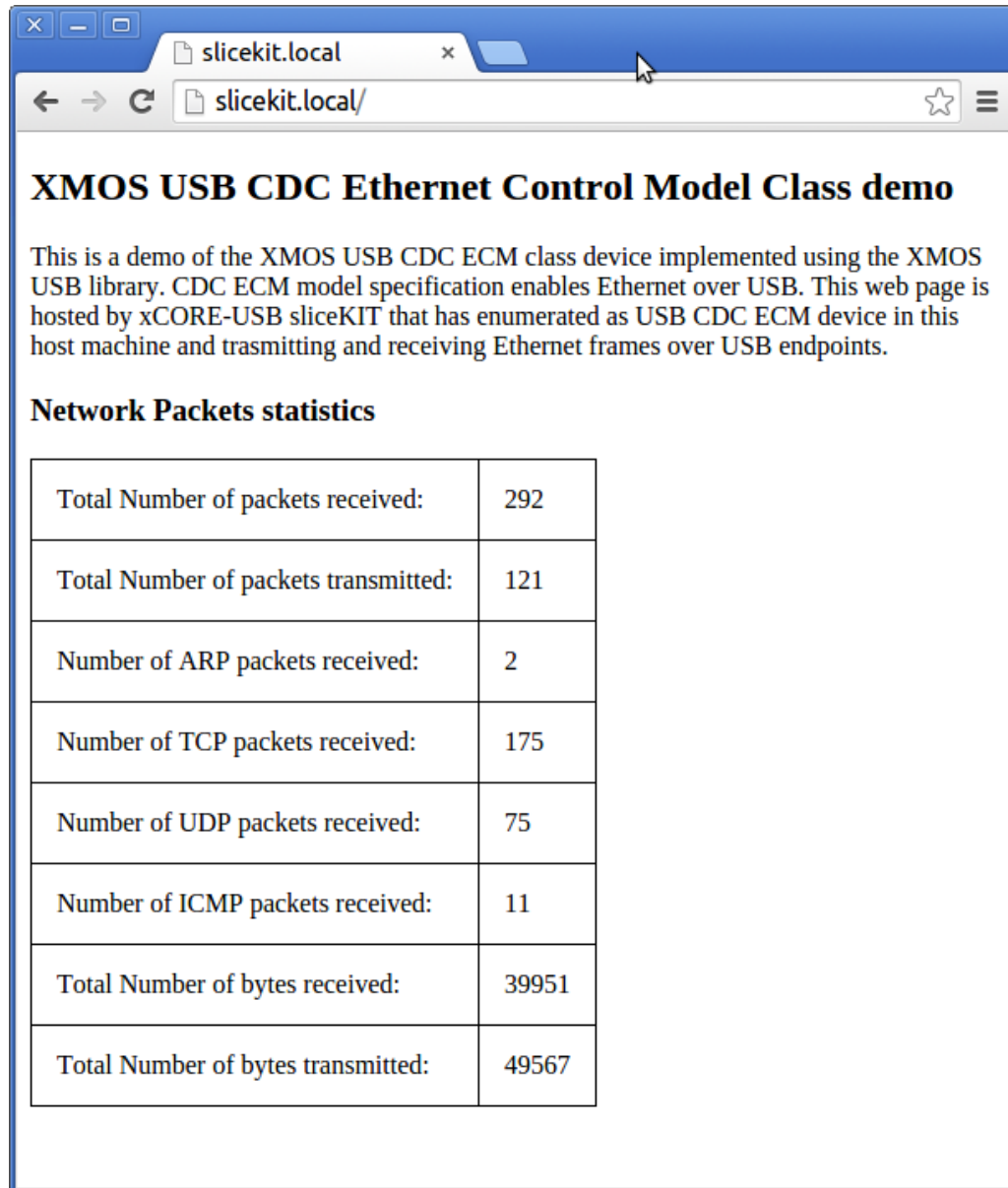


Figure 9: Webpage hosted by USB CDC-ECM device

- Refresh the opened webpage to see the packets count updated. Also note the number of ICMP packets is same as the number of ping requests sent out previously.

APPENDIX C - References

XMOS Tools User Guide

<http://www.xmos.com/published/xtimecomposer-user-guide>

XMOS xCORE Programming Guide

<http://www.xmos.com/published/xmos-programming-guide>

XMOS xCORE-USB Device Library

<http://www.xmos.com/published/xuddg>

XMOS USB Device Design Guide

<http://www.xmos.com/published/xmos-usb-device-design-guide>

USB CDC Class Specification, USB.org:

http://www.usb.org/developers/docs/devclass_docs/

USB 2.0 Specification

http://www.usb.org/developers/docs/usb20_docs/usb_20_081114.zip

Ethernet Frame Format

http://en.wikipedia.org/wiki/Ethernet_frame

APPENDIX D - Full source code listing

D.1 Source code for main.xc

```
// Copyright (c) 2016, XMOS Ltd, All rights reserved

#include <platform.h>
#include <xs1.h>
#include <xscope.h>
#include <string.h>
#include <stdio.h>
#include <timer.h>
#include "usb.h"
#include "ethernet.h"

/* xSCOPE Setup Function */
#if (USE_XSCOPE == 1)
void xscope_user_init(void) {
    xscope_register(0, 0, "", 0, "");
    xscope_config_io(XSCOPE_IO_BASIC); /* Enable fast printing over XTAG */
}
#endif

/* USB Endpoint Defines */
#define XUD_EP_COUNT_OUT 2 //Includes EP0 (1 OUT EP0 + 1 BULK OUT EP)
#define XUD_EP_COUNT_IN 3 //Includes EP0 (1 IN EP0 + 1 INTERRUPT IN EP + 1 BULK IN EP)

int main() {
    chan c_ep_out[XUD_EP_COUNT_OUT], c_ep_in[XUD_EP_COUNT_IN];
    interface usb_cdc_ecm_if cdc_ecm;

    /* 'Par' statement to run the following tasks in parallel */
    par
    {
        on USB_TILE: xud(c_ep_out, XUD_EP_COUNT_OUT, c_ep_in, XUD_EP_COUNT_IN,
            null, XUD_SPEED_HS, XUD_PWR_SELF);

        on USB_TILE: Endpoint0(c_ep_out[0], c_ep_in[0]);

        on USB_TILE: CdcEcmEndpointsHandler(c_ep_in[1], c_ep_out[1], c_ep_in[2], cdc_ecm);

        on USB_TILE: EthernetFrameHandler(cdc_ecm);
    }
    return 0;
}
```

D.2 Source code for xud_ecm.xc

```
// Copyright (c) 2016, XMOS Ltd, All rights reserved

#include <xs1.h>
#include <stdio.h>
#include <string.h>
#include "packet_buffer.h"
#include "queue.h"
#include "usb.h"
#include "xud_ecm.h"
#include "ethernet.h"

/* USB CDC device product defines */
#define BCD_DEVICE 0x0100
#define VENDOR_ID 0x20B1
#define PRODUCT_ID 0x0402

/* USB Sub class and protocol codes */
#define USB_CDC_ECM_SUBCLASS 0x06

/* CDC interface descriptor type */
#define USB_DESCTYPE_CS_INTERFACE 0x24
```

```
#define USB_DESCRIPTOR_CS_ENDPOINT    0x25

/* Endpoint Addresses for CDC device */
#define CDC_NOTIFICATION_EP_NUM      1 /* (0x81) */
#define CDC_DATA_RX_EP_NUM           1 /* (0x01) */
#define CDC_DATA_TX_EP_NUM           2 /* (0x82) */

/* Data endpoint packet size */
#define MAX_EP_SIZE                    512

/* CDC ECM Class requests Section 6.2 in CDC ECM spec */
#define SET_ETHERNET_MULTICAST_FILTERS    0x40
#define SET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER 0x41
#define GET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER 0x42
#define SET_ETHERNET_PACKET_FILTER       0x43
#define GET_ETHERNET_STATISTIC            0x44
/* 45h-4Fh RESERVED (future use) */

/* CDC ECM Class notification codes - Section 6.3 in CDC ECM spec */
#define NETWORK_CONNECTION              0x00
#define RESPONSE_AVAILABLE              0x01
#define CONNECTION_SPEED_CHANGE        0x2A

/* Definition of Descriptors */
/* USB Device Descriptor */
static unsigned char devDesc[] =
{
    0x12,          /* 0 bLength */
    USB_DESCRIPTOR_DEVICE, /* 1 bDescriptorType - Device */
    0x00,          /* 2 bcdUSB version */
    0x02,          /* 3 bcdUSB version */
    USB_CLASS_COMMUNICATIONS, /* 4 bDeviceClass - USB CDC Class */
    0x00,          /* 5 bDeviceSubClass - Specified by interface */
    0x00,          /* 6 bDeviceProtocol - Specified by interface */
    0x40,          /* 7 bMaxPacketSize for EP0 - max = 64 */
    (VENDOR_ID & 0xFF), /* 8 idVendor */
    (VENDOR_ID >> 8), /* 9 idVendor */
    (PRODUCT_ID & 0xFF), /* 10 idProduct */
    (PRODUCT_ID >> 8), /* 11 idProduct */
    (BCD_DEVICE & 0xFF), /* 12 bcdDevice */
    (BCD_DEVICE >> 8), /* 13 bcdDevice */
    0x01,          /* 14 iManufacturer - index of string */
    0x02,          /* 15 iProduct - index of string */
    0x00,          /* 16 iSerialNumber - index of string */
    0x01,          /* 17 bNumConfigurations */
};

/* USB Configuration Descriptor */
static unsigned char cfgDesc[] = {

    0x09,          /* 0 bLength */
    USB_DESCRIPTOR_CONFIGURATION, /* 1 bDescriptorType - Configuration */
    0x47, 0x00,    /* 2 wTotalLength */
    0x02,          /* 4 bNumInterfaces */
    0x01,          /* 5 bConfigurationValue */
    0x04,          /* 6 iConfiguration - index of string */
    0x80,          /* 7 bmAttributes - Bus powered */
    0xC8,          /* 8 bMaxPower - 400mA */

    /* CDC Communication interface */
    0x09,          /* 0 bLength */
    USB_DESCRIPTOR_INTERFACE, /* 1 bDescriptorType - Interface */
    0x00,          /* 2 bInterfaceNumber - Interface 0 */
    0x00,          /* 3 bAlternateSetting */
    0x01,          /* 4 bNumEndpoints */
    USB_CLASS_COMMUNICATIONS, /* 5 bInterfaceClass */
    USB_CDC_ECM_SUBCLASS, /* 6 bInterfaceSubClass - Ethernet Control Model */
    0x00,          /* 7 bInterfaceProtocol - No specific protocol */
    0x00,          /* 8 iInterface - No string descriptor */

    /* Header Functional descriptor */
    0x05,          /* 0 bLength */
    USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
    0x00,          /* 2 bDescriptorSubtype, HEADER */

```

```

0x10, 0x01,          /* 3 bcdCDC */

/* Union Functional descriptor */
0x05,                /* 0 bLength */
USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
0x06,                /* 2 bDescriptorSubtype, UNION */
0x00,                /* 3 bControlInterface - Interface 0 */
0x01,                /* 4 bSubordinateInterface0 - Interface 1 */

/* Ethernet Networking Functional descriptor */
0x0D,                /* 0 bLength - 13 bytes */
USB_DESCRIPTOR_CS_INTERFACE, /* 1 bDescriptorType, CS_INTERFACE */
0x0F,                /* 2 bDescriptorSubtype, ETHERNET_NETWORKING */
0x03,                /* 3 iMACAddress, Index of MAC address string */
0x00, 0x00, 0x00, 0x00, /* 4 bmEthernetStatistics - Handles None */
0xEA, 05,           /* 8 wMaxSegmentSize - 1514 bytes */
0x00, 0x00,         /* 10 wNumberMCFilters - No multicast filters */
0x00,               /* 12 bNumberPowerFilters - No wake-up feature */

/* Notification Endpoint descriptor */
0x07,                /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
(CDC_NOTIFICATION_EP_NUM | 0x80), /* 2 bEndpointAddress - IN endpoint */
0x03,                /* 3 bmAttributes - Interrupt type */
0x40,                /* 4 wMaxPacketSize - Low */
0x00,                /* 5 wMaxPacketSize - High */
0xFF,               /* 6 bInterval */

/* CDC Data interface */
0x09,                /* 0 bLength */
USB_DESCRIPTOR_INTERFACE, /* 1 bDescriptorType */
0x01,                /* 2 bInterfaceNumber */
0x00,                /* 3 bAlternateSetting */
0x02,                /* 4 bNumEndpoints */
USB_CLASS_CDC_DATA, /* 5 bInterfaceClass */
0x00,                /* 6 bInterfaceSubClass */
0x00,                /* 7 bInterfaceProtocol */
0x00,                /* 8 iInterface - No string descriptor */

/* Data OUT Endpoint descriptor */
0x07,                /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
CDC_DATA_RX_EP_NUM, /* 2 bEndpointAddress - OUT endpoint */
0x02,                /* 3 bmAttributes - Bulk type */
0x00,                /* 4 wMaxPacketSize - Low */
0x02,                /* 5 wMaxPacketSize - High */
0x00,                /* 6 bInterval */

/* Data IN Endpoint descriptor */
0x07,                /* 0 bLength */
USB_DESCRIPTOR_ENDPOINT, /* 1 bDescriptorType */
(CDC_DATA_TX_EP_NUM | 0x80), /* 2 bEndpointAddress - IN endpoint */
0x02,                /* 3 bmAttributes - Bulk type */
0x00,                /* 4 wMaxPacketSize - Low byte */
0x02,                /* 5 wMaxPacketSize - High byte */
0x01,                /* 6 bInterval */
};

unsafe{
/* String table - unsafe as accessed via shared memory */
static char * unsafeStringDescriptors[] =
{
    "\x09\x04",          /* Language ID string (US English) */
    "XMOS",              /* iManufacturer */
    "XMOS USB CDC Ethernet Device", /* iProduct */
    "002297000000",      /* iMACAddress */
    "Config",            /* iConfiguration string */
};
}

#define MAC_ID_INDEX 3

/* Queues for handling Ethernet frame buffers */
Queue_t toHostQ;
Queue_t toDevQ;

```

```

/* CDC Class-specific requests handler function */
XUD_Result_t ControlInterfaceClassRequests(XUD_ep ep_out, XUD_ep ep_in, USB_SetupPacket_t sp)
{
    XUD_Result_t result = XUD_RES_ERR;

    #if defined (DEBUG) && (DEBUG == 1)
        printhexln(sp.bRequest);
    #endif

    switch(sp.bRequest)
    {
        case SET_ETHERNET_MULTICAST_FILTERS:
            /* Not supported now */

            /*if((result = XUD_GetBuffer(ep_out, (buffer, unsigned char[]), 0)) != XUD_RES_OKAY)
            {
                return result;
            }

            result = XUD_DoSetRequestStatus(ep_in);*/

            return result;
            break;

        case SET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER:
            /* Not supported now */

            /*if((result = XUD_GetBuffer(ep_out, (buffer, unsigned char[]), length)) != XUD_RES_OKAY)
            {
                return result;
            }

            result = XUD_DoSetRequestStatus(ep_in);*/
            return result;
            break;

        case GET_ETHERNET_POWER_MANAGEMENT_PATTERN_FILTER:
            /* Not supported now */

            /* return XUD_DoGetRequest(ep_out, ep_in, (buffer, unsigned char[]), 0, sp.wLength);
            */
            return result;
            break;

        case SET_ETHERNET_PACKET_FILTER:
            /* TODO: Handle this required command */

            return XUD_DoSetRequestStatus(ep_in);

            break;

        case GET_ETHERNET_STATISTIC:
            /* Not supported now */

            /* return XUD_DoGetRequest(ep_out, ep_in, (buffer, unsigned char[]), 0, sp.wLength); */

            break;

        default:
            // Error case
            printhexln(sp.bRequest);
            break;
    }
    return XUD_RES_ERR;
}

/* Endpoint 0 handling both std USB requests and CDC class specific requests */
void Endpoint0(chanend chan_ep0_out, chanend chan_ep0_in)
{
    USB_SetupPacket_t sp;

    unsigned bmRequestType;
    XUD_BusSpeed_t usbBusSpeed;

```

```

XUD_ep ep0_out = XUD_InitEp(chan_ep0_out, XUD_EPTYPE_CTL | XUD_STATUS_ENABLE);
XUD_ep ep0_in = XUD_InitEp(chan_ep0_in, XUD_EPTYPE_CTL | XUD_STATUS_ENABLE);

/* Get MAC ID from a global variable */
getMacAddressString(stringDescriptors[MAC_ID_INDEX]);

while(1)
{
    /* Returns XUD_RES_OKAY on success */
    XUD_Result_t result = USB_GetSetupPacket(ep0_out, ep0_in, sp);

    if(result == XUD_RES_OKAY)
    {
        /* Set result to ERR, we expect it to get set to OKAY if a request is handled */
        result = XUD_RES_ERR;

        /* Stick bmRequest type back together for an easier parse... */
        bmRequestType = (sp.bmRequestType.Direction<<7) |
            (sp.bmRequestType.Type<<5) |
            (sp.bmRequestType.Recipient);

        if ((bmRequestType == USB_BMREQ_H2D_STANDARD_DEV) &&
            (sp.bRequest == USB_SET_ADDRESS))
        {
            // Host has set device address, value contained in sp.wValue
        }

        switch(bmRequestType)
        {
            /* Direction: Device-to-host and Host-to-device
             * Type: Class
             * Recipient: Interface
             */
            case USB_BMREQ_H2D_CLASS_INT:
            case USB_BMREQ_D2H_CLASS_INT:

                /* Inspect for CDC Communications Class interface num */
                if(sp.wIndex == 0)
                {
                    /* Returns XUD_RES_OKAY if handled,
                     * XUD_RES_ERR if not handled,
                     * XUD_RES_RST for bus reset */
                    result = ControlInterfaceClassRequests(ep0_out, ep0_in, sp);
                }
                break;
        }
    } /* if ends */

    /* If we haven't handled the request about then do standard enumeration requests */
    if(result == XUD_RES_ERR )
    {
        /* Returns XUD_RES_OKAY if handled okay,
         * XUD_RES_ERR if request was not handled (STALLED),
         * XUD_RES_RST for USB Reset */
        unsafe{
            result = USB_StandardRequests(ep0_out, ep0_in, devDesc,
                sizeof(devDesc), cfgDesc, sizeof(cfgDesc),
                null, 0, null, 0, stringDescriptors, sizeof(stringDescriptors)/sizeof(
                    ↪ stringDescriptors[0]),
                sp, usbBusSpeed);
        }
    }

    /* USB bus reset detected, reset EP and get new bus speed */
    if(result == XUD_RES_RST)
    {
        usbBusSpeed = XUD_ResetEndpoint(ep0_out, ep0_in);
    }
}

/* Function to handle all endpoints of the CDC class excluding control endpoint0 */
void CdcEcmEndpointsHandler(chanend c_epint_in, chanend c_epbulk_out, chanend c_epbulk_in, SERVER_INTERFACE(
    ↪ usb_cdc_ecm_if, cdc_ecm))
{

```

```

int inBufId = 0, outBufId = 0;          // used to identify buffer read/write by device
int inIndex = 0, outLen = 0;
int hostWaiting = 1, devWaiting = 0; // Used for flow control with IN and OUT endpoints respectively
unsigned length;
XUD_Result_t result;

/* Initialize all endpoints */
XUD_ep epint_in = XUD_InitEp(c_epint_in, XUD_EPTYPE_INT);
XUD_ep epbulk_out = XUD_InitEp(c_epbulk_out, XUD_EPTYPE_BUL);
XUD_ep epbulk_in = XUD_InitEp(c_epbulk_in, XUD_EPTYPE_BUL);

/* XUD will NAK if the endpoint is not ready to communicate with XUD */

/* TODO: Interrupt endpoint to report network state (if required) */

/* Just to keep compiler happy */
epint_in = epint_in;

/* Initialize the Queues and Buffers */
qInit(toHostQ);
qInit(toDevQ);
packetBufferInit();

/* Get a free buffer to set OUT endpoint ready for receiving data */
outBufId = packetBufferAlloc();
outLen = 0;
XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[])); // Re-interpret int buffer to
    ↳ char buffer

while(1)
{
    select
    {
        case XUD_GetData_Select(c_epbulk_out, epbulk_out, length, result):
            if(result == XUD_RES_OKAY)
            {
                /* Received some data */
                if(length < MAX_EP_SIZE) {
                    /* USB Short packet or Zero length packet is received */
                    outLen += length;
                    /* Ethernet packet is received completely */
                    qPut(toDevQ, outBufId, outLen);

                    if(qIsFull(toDevQ)) {
                        devWaiting = 1;
                    } else {
                        outBufId = packetBufferAlloc();
                        outLen = 0;
                        XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[]));
                    }
                } else {
                    /* USB Full packet */
                    outLen += MAX_EP_SIZE;
                    XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[])+outLen);
                }
            } else {
                XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[])+outLen);
            }
            break;

        case XUD_SetData_Select(c_epbulk_in, epbulk_in, result):
            /* USB Packet sent successfully when result is XUD_RES_OKAY */
            int index = qPeek(toHostQ);
            int bytesSent = toHostQ.data[index].from - inIndex;
            inIndex = toHostQ.data[index].from;
            int bytesToSend = toHostQ.data[index].len - toHostQ.data[index].from;

            if(bytesToSend) {
                if (bytesToSend > MAX_EP_SIZE) {
                    /* Still Large packet, split the transfer */
                    bytesToSend = MAX_EP_SIZE;
                }
            }
    }
}

```



```

    }
    XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[])+inIndex, bytesToSend);
    toHostQ.data[index].from += bytesToSend;

} else if (bytesSent == MAX_EP_SIZE){
    /* Send a Zero Length Packet to indicate completion of transfer */
    XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[]), 0);
    inIndex += bytesSent;
} else {
    /* Ethernet frame transfer is over */
    packetBufferFree(toHostQ.data[index].packet);
    /* Remove packet out of queue */
    qGet(toHostQ);

    /* Check if other packet is waiting to be sent to host */
    if(!qIsEmpty(toHostQ)) {
        index = qPeek(toHostQ);
        inBufId = toHostQ.data[index].packet;
        inIndex = toHostQ.data[index].from;
        bytesToSend = toHostQ.data[index].len;

        if(bytesToSend > MAX_EP_SIZE) {
            bytesToSend = MAX_EP_SIZE;
        }
        XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[])+inIndex, bytesToSend);
        toHostQ.data[index].from += bytesToSend;
    } else {
        /* No packets are available to send */
        hostWaiting = 1;
    }
}
break;

/* Case handlers for CDC ECM interface functions */
case (!qIsEmpty(toDevQ)) => cdc_ecm.read_frame(unsigned char frame_buf[], REFERENCE_PARAM(unsigned,
↪ frame_len)):

    /* A packet waiting in queue to be handled */
    int index = qPeek(toDevQ);
    int packetNum = toDevQ.data[index].packet;
    int offset = toDevQ.data[index].from;
    frame_len = toDevQ.data[index].len;

    /* Copy the whole Ethernet frame */
    memcpy(frame_buf, (packetBuffer[packetNum], unsigned char[])+offset, frame_len);
    packetBufferFree(packetNum);
    qGet(toDevQ);

    /* Make a buffer ready for OUT endpoint */
    if(devWaiting) {
        outBufId = packetBufferAlloc();
        XUD_SetReady_Out(epbulk_out, (packetBuffer[outBufId], unsigned char[]));
        devWaiting = 0;
    }
    break;

case (!qIsFull(toHostQ)) => cdc_ecm.send_frame(unsigned char frame_buf[], REFERENCE_PARAM(unsigned,
↪ frame_len)):

    /* Get a free buffer to write the frame */
    int buf_id = packetBufferAlloc();
    memcpy((packetBuffer[buf_id], unsigned char[]), frame_buf, frame_len);
    qPut(toHostQ, buf_id, frame_len);

    /* Send the packet if host is waiting */
    if(hostWaiting) {
        int index = qPeek(toHostQ);
        inBufId = toHostQ.data[index].packet;
        inIndex = toHostQ.data[index].from;
        int bytesToSend = toHostQ.data[index].len;

        hostWaiting = 0;

        if(bytesToSend > MAX_EP_SIZE ){
            /* Large packet, split the transfers */

```

```
        bytesToSend = MAX_EP_SIZE;
    }
    XUD_SetReady_In(epbulk_in, (packetBuffer[inBufId], unsigned char[])+inIndex, bytesToSend);
    toHostQ.data[index].from += bytesToSend;
}
break;

case cdc_ecm.is_frame_available() -> int val:
    val = !qIsEmpty(toDevQ);
    break;
}
}
```

